Context Purification for Cell-Level Error Detection with Tabular Foundation Models

Ngo Dang Trong Hieu[0009−0007−1773−3680]

Ton Duc Thang University
ngodangtronghieu.dl2021@gmail.com

Abstract. Conditional prediction can score individual cells of a dirty table, but the reference examples used by a tabular foundation model can themselves be corrupted. We evaluate a fixed purification rule that removes context rows with the largest nested out-of-fold target-cell scores, with all first-pass work isolated from the outer query fold. A paired random-removal control holds base contexts and removal counts fixed, and nested preparation costs are reported explicitly. The study covers four capped benchmark tables, real and two injected-error conditions, five seeds, and both TabPFN-2 and LightGBM. On real errors, purification was worse than matched random removal on three tables and better on one (TabPFN AP difference -0.161 on hospital, -0.049 on flights, $+0.026$ on beers and -0.030 on rayyan), and no detector, including the simple baselines, was best on every table. These are per-table findings with conditional row-bootstrap uncertainty, not a population-wide verdict. We also report support, failures, rare-valid alarms, boundary sensitivity and compute on the complete eligible-cell universe. The results argue for testing context selection against matched random removal rather than assuming that a cleaner-looking context helps.

Keywords: Cell error detection · Tabular foundation models · Context purification · Contaminated context.

1 Introduction

Data errors can affect only a few attributes of a record. A suspicious product measurement does not imply that its name and producer are wrong, and a rare bibliographic entry need not contain an error. This makes cell localization a different objective from ranking entire rows as anomalies. Robust cellwise methods already exploit relationships among variables [16]; a foundation model does not make that problem new. The question addressed here is whether the context used by a conditional predictor can be improved without access to clean examples or error labels.

A pretrained tabular foundation model (TFM) offers a practical route to conditional scoring. Treat each column in turn as a prediction target, use the remaining columns as features, and score the observed target value under the resulting prediction. TabPFN supports classification and regression through in-context

prediction [5]. In a dirty table, however, the reference examples are themselves imperfect. Repeated corrupt values may become predictable, while infrequent correct values can receive large suspicion scores. Removing the highest-scoring context cells' rows could therefore either improve the reference distribution or erase legitimate variation. Neither outcome follows from the model's supervised prediction capability.

We study this intervention under a controlled cell-detection protocol. For each target column and outer query fold, a nested first pass scores only the outer training pool. The purification arm removes context rows with the largest target-cell scores. A random arm removes exactly the same number of rows from the same base context. Both retain the same queries and evaluation denominator, which separates targeted selection from simply reducing context size; the extra first-pass computation is reported separately.

The empirical scope is deliberately specific: four capped benchmark tables, their real errors, two controlled corruption conditions and five seeds. Real-error purification is primary; injections and an observed-key leave-entity-out analysis are probes. Effects are reported per source table; repeated conditions and seeds are not treated as additional independent datasets.

The study contributes an outer-fold-isolated implementation of a simple context intervention, a matched-count and explicitly costed comparison against random removal, and an artifact-backed assessment spanning accuracy, rare-valid alarms, support, failures and boundary sensitivity. It does not introduce a new pretrained model, an influence estimator, or a conformal guarantee.

2 Related Work and Scope

Cellwise detection and imputation. DDC uses robust relationships between numerical variables to predict cells and assess their residuals [16]. Its published algorithm includes robust standardization, bivariate relations, prediction and deshrinkage. Our numerical DDCcore baseline is a simpler comparator and cannot establish a ranking against reference DDC. CellGMM addresses numerical populations with a mixture model and a detection–imputation step [17]; discarding an entire record can lose useful information from its other cells. Our removal operates on whole context rows for one target at a time and offers no corresponding mixture-estimation or cell-repair guarantee. Cellwise trimming has also been built into model-based co-clustering [4], so trimming as such is not new here. Conformal Data Cleaning (CDC) combines column prediction with calibrated prediction sets or intervals [7]. Its evaluated deployment scenario trains on clean data and subsequently encounters corrupted inputs. Our deployment arms receive one dirty table and no clean calibration split. The LightGBM conditional comparator is inspired by column prediction but does not reproduce CDC, and our scores have no conformal coverage interpretation.

Foundation models and contaminated contexts. TabPFN provides pretrained conditional prediction for small tabular problems [5]. We use pinned second-generation weights through an explicit adapter, rather than whichever model

a current package would select by default. Unsupervised extensions of TabPFN also use conditional prediction; we do not claim this reduction as novel or call our raw arm a reproduction of the extensions. TACTIC explicitly studies contaminated contexts for row anomaly detection [11]. Its anomaly-oriented pretraining and row-level evaluation address a related robustness problem. Consequently, context contamination itself is established prior work; our intervention concerns cell-level rankings under fixed pretrained weights and nested selection, not a row-AUROC comparison.

Data quality and context selection for TFMs. QualiTab evaluates imperfect data and contrasts perfect-data, perfect-context and no-intervention settings for TabPFN embeddings [14]. Its clean-context comparison is close to the motivation of our oracle diagnostic; the benefit of cleaner contexts is therefore not a new hypothesis introduced here. Model-aware cleaning learns reinforcement-learning policies that sequence cleaning operators for TFM-oriented data preparation [2]. We instead evaluate one fixed removal rule as a cell-ranking intervention without learning a cleaning policy. LUCoS selects representative contexts using unsupervised PFN embeddings, then obtains labels for supervised prediction [6]. Our contexts contain observed target values from the dirty table, with no annotation oracle for deployment. A study of test-time compute for TFMs reports that attention-guided row retrieval helps on some large tables but can fail badly, for example by distorting class composition, and that context expansion gives no consistent gain [13]. It concerns supervised prediction from labelled contexts, not detecting errors in a dirty context, so it is related evidence rather than a direct comparator. Together these works narrow what can be claimed as new: TFM-oriented data cleaning and label-free context selection already have precedents, so our contribution is limited to the cell-level, dirty-context setting studied here.

Systems for heterogeneous error detection. Raha is a configuration-free error-detection system and supplies the benchmark repository used here [10]. Auto-Test learns semantic-domain constraints from a large table corpus offline and applies them to detect erroneous column values without per-table supervision [3]. MaTElDa is semi-supervised: it groups cells of several tables by domain and quality similarity so that a small labelling budget can be shared across tables [1]. LUCARIO targets unsupervised, lightweight detection of pattern anomalies and outliers [15]. These are direct cell-level competitors with different knowledge sources (a table corpus, user labels, inferred constraints); none was run under our protocol. We use its paired dirty/clean tables, not a reimplementation of its complete detection workflow. ZeroED combines LLM-derived representations and labels with detector training [12]. That information and compute regime differs from our label-free local deployment arms. Neither system's published headline score is an interchangeable baseline result for our capped-table protocol.

Measured comparators. We run frequency and pattern scores, a robust numerical score, column-wise Isolation Forest [9], DDCcore and conditional LightGBM [8]. They test whether marginal rarity, simple numerical deviation or the choice of

conditional model explains an effect; they cannot rank the declared rule against every context-selection strategy.

3 Conditional Scoring and Context Purification

3.1 Task and information boundary

Let $\widetilde{X}$ be an observed table with n rows and d eligible columns. The detector outputs $S \in \mathbb{R}^{n \times d}$, with larger values indicating greater suspicion. Evaluation compares observed strings with aligned clean strings; this defines benchmark disagreement, rather than a universal semantic definition of an error. Clean values and the error matrix are unavailable to deployment arms. Only a separately labelled oracle receives clean contexts.

For seed s, partition rows into $K = 5$ outer folds Q_f. Sample a sorted base context B_f of at most $C = 512$ rows from $\{1, \ldots, n\} \setminus Q_f$. Share these bases across target columns, removal arms and models. To score column j, drop j from the features and predict its observed values from the remaining eligible columns. Excluded identifier columns are not conditioning features. The query fold never enters a fitted deployment context. Other rows of the same entity may be used in the primary row-level analysis.

Typing and feature encoding are fitted on the complete observed table for the final pass. Numerical features use parsed numbers with unparseable entries as missing; other features use observed frequency-rank codes and an unknown code. This is label-free transductive preprocessing: outer queries can affect encoding, although they do not enter the model's fitted context. We distinguish this assumption from a fully inductive fit on context rows only.

3.2 Mixed-type suspicion scores

A column is numerical if at least 80% of its nonempty observed strings parse as finite numbers. Otherwise it is categorical, unless its observed unique-value ratio exceeds 0.5, in which case the target is unsupported. Unsupported cells receive score zero and remain in every whole-universe evaluation denominator. They may still supply encoded conditioning features.

For a parseable numerical query y and context of size b, let b_p be the number of parseable context targets. With predictive CDF F, the score is

$$S(y) = -\log \frac{b_p + 0.5}{b + 1} - \log \max\{2 \min(F(y), 1 - F(y)), 10^{-12}\}. \tag{1}$$

The second term is a two-sided tail score, not a probability density. With fewer than ten parseable context targets or no numerical range, a smoothed exact-match frequency replaces that term. Unparseable query tokens use a smoothed nonparseability probability followed by their smoothed frequency among unparseable context targets. These rules retain empty strings and formatting variants rather than repairing them before detection.

For a categorical target with at most ten context classes, the model predicts class probabilities. With more classes, retain the nine most frequent and pool the others into an OTHER class. Divide OTHER mass by smoothed within-pool frequencies, allowing probability for an unseen query token. If the context contains a single class, use the degenerate class distribution. A final additive smoothing term assigns mass to values absent from the context:

$$S(y) = -\log\left[\frac{b\widehat{p}(y \mid x_{-j}) + 1/(v+1)}{b+1}\right], \tag{2}$$

where v is the number of distinct observed context target values and $\widehat{p}$ includes OTHER redistribution when applicable. Previously unseen tokens share an unknown bucket; this is not a probability model over every possible unseen string. These numerical and categorical scores share an orientation but have no proven common calibration. We therefore retain type-specific analyses alongside pooled AP.

Relation to residual and conformal scores. DDC standardizes residuals from robust linear relations among numerical variables [16]; our scores instead come from the predictive distribution of a pretrained or task-fitted model, without a linearity assumption. CDC calibrates prediction sets on clean data [7]. Our deployment arms have no clean calibration split and the context may contain errors, so S is used only as an uncalibrated ranking evaluated by precision–recall metrics. Removal is based on first-pass suspicion scores, not on a gradient-based influence estimate.

3.3 Outer-isolated selection

The purification parameter is fixed at $q = 0.15$. For each outer fold, score every row of its training pool by an inner raw cross-fit. Inner typing, encoding, contexts and fits all use that pool exclusively. Let $U_{ij}^{(f)}$ be the inner out-of-fold score for target j. For each base B_f, remove the $r_f = \text{round}(q|B_f|)$ rows with largest $U_{ij}^{(f)}$, breaking ties by increasing row index. Fit the final conditional model on the remaining dirty context and score Q_f. Selection is column specific: the rows removed for one target need not be removed for another.

Reusing a raw score matrix computed on the entire table would break this isolation: training examples used to score a prospective context row could include the eventual outer query. The nested construction prevents that dependency. It does not prove that the highest scores identify corrupted contexts, nor that removing them preserves rare correct values.

3.4 Controls, diagnostics and compute

Raw retains B_f. *Random* removes r_f rows without replacement using a separate deterministic stream for each seed, fold and target. *Oracle* keeps the same base IDs but substitutes clean target and feature values in the context; query values

Outer-isolated purification for seed s
1. Construct row folds Q_f and shared bases B_f.
2. For each outer fold f, form $P_f = \{1, \ldots, n\} \setminus Q_f$.
3. Inner-cross-fit raw scores $U^{(f)}$ using only $\widetilde{X}[P_f]$.
4. For each target j, rank B_f by $U_{ij}^{(f)}$ and remove r_f rows.
5. Fit on the retained dirty context, omitting feature j; score Q_f.
6. Retain unsupported cells and failure masks in the full output universe.

Fig. 1. Selection has no dependency on the outer query through the first pass. Final encoding remains explicitly transductive. This is one trimming step, not an iterative procedure or a gradient-based influence estimate.

remain observed and the encoder remains dirty fitted. Its gap is diagnostic and need not be positive. *Leaky*, on real-error conditions only, unions the query fold with the base; its intentional violation of exclusion is reported separately.

The secondary *raw-LEO* arm constructs folds of observed entity keys, using provider number, flight and brewery identifier on their declared tables. No observed query key enters a fitted context. Corrupted keys can split a true entity and shared or empty keys can merge entities. Bases differ from row-level bases, and the final encoder is still transductive; this arm is a boundary sensitivity analysis rather than proof of independence.

Purification requires up to K^2d additional column tasks for nested scoring, besides up to Kd final tasks. The corresponding number for a single raw or random arm is at most Kd; degenerate or unsupported targets can bypass a model fit, while LightGBM residual calibration can require additional fits. These are task-count bounds, not measured neural forward-call counts. We measure nested preparation and inclusive purification time separately. The accounted matched-compute random cost is ordinary random time plus the same nested preparation time, using unchanged random scores; it is not a separately timed arm. No task-specific TabPFN weight update is performed; its context fitting and preprocessing remain chargeable work. LightGBM, in contrast, fits task-specific trees for each context.

4 Experimental Protocol

4.1 Data, roles and corruption conditions

We use a pinned snapshot of the Raha benchmark repository, upstream commit `7be1334b8c7bbdac3f47ef514fb3e1e8c5fc181c`, with per-file hashes. All CSV values are read as strings, retaining empty strings. An eligible cell is an entry outside a uniquely valued column named `id`, `index` or `tuple_id`, with eligibility established on the source observed table. The error label is an exact string disagreement with the aligned reference. Unsupported targets remain eligible; successful-output-only evaluation would otherwise reward methods for avoiding difficult columns.

`movies_1` is development data and does not enter confirmatory summaries. The confirmatory collection comprises `hospital`, `flights`, `beers` and `rayyan`. Each table is capped at 1,000 rows before injection: tables below the cap retain every row; larger tables use a single seed-0 sample without replacement, sorted in original row order. Identical source rows and column order are used across conditions, models and seeds. The study targets these capped tables, not their uncapped distributions.

Each table has three conditions. *Real* retains the observed values. *Random injection* starts from the clean reference and attempts corruption at individually sampled eligible positions. *Systematic injection* also starts from the reference but repeats a shared wrong value across sampled occurrences of the same column value. Both target a 10% corruption rate. Attempted error types are sampled uniformly from character edits, replacement by another column value, empty-string replacement, format changes and numeric rescaling. Rejected or unchanged proposals imply that realized type proportions are not uniform. A fixed attempt guard could produce a shortfall, but all 40 injected units reached exactly the 10% target. These are string-level mechanism probes, not claims to reproduce every natural error process.

Run seeds are $0, 1, 2, 3, 4$. The injection and context algorithms both use the run seed; their randomness is coupled rather than independently crossed. The primary contrast averages the five paired AP differences separately for each real table. Corruption conditions remain separate repeated measurements within a source table.

4.2 Models and comparators

TabPFN uses package version 9.2.0 and explicit second-generation classifier and regressor checkpoint paths. The pinned upstream revisions are recorded with checkpoint and license hashes in the reproducibility manifest. We use one estimator, CPU inference and the run seed for model randomness. Package and weight licenses are evaluated separately; later package defaults or checkpoint generations are not substituted.

The conditional LightGBM model uses 100 trees, learning rate 0.1, 15 leaves, minimum child size five and one training thread. Classification predicts the context target classes. Regression uses a Gaussian tail about its predicted mean, with scale estimated from within-context cross-validated residuals. Thus the conditional-model comparison includes different distributional approximations, not just a backbone replacement with equivalent calibration.

Value frequency (VF) scores smoothed leave-one-out token frequency in each column. Pattern frequency (PF) applies this rule to character-class signatures. RobustZ scores numerical deviations using median and MAD with the declared fallback; nonnumerical columns receive zero. ColumnIF fits 100 Isolation Forest trees separately to each column, using numbers or frequency encodings. DDCcore combines robust numerical standardization, selected pairwise relationships and residual scores. It omits parts of reference DDC and is reported under its own

name. All methods share the full eligible denominator, while their support sets may differ.

4.3 Endpoints and uncertainty

Average precision (AP) is the primary metric, with tied scores grouped as in `average_precision_score`; it is not trapezoidal PR integration. For real table t, the primary estimand is

$$\Delta_t = \frac{1}{5}\sum_{s=0}^{4}\big[\mathrm{AP}_{t,s}(\mathrm{TFM},\mathrm{purify}) - \mathrm{AP}_{t,s}(\mathrm{TFM},\mathrm{random})\big]. \tag{3}$$

AP is calculated per seed before taking differences and means. We also report R-precision, precision at 1% and 5% audit budgets, numerical/categorical AP, supported-column summaries, finite-output coverage and failure counts. Exact top-k ties use stable row-major order. R-precision uses the reference error count as an evaluation-only budget, not an error count known at deployment. Type-specific summaries follow the declared numerical/categorical target groups; unsupported text remains visible in the whole-universe and support reports.

Rare-valid cells are reference-correct entries whose clean marginal value frequency is at most 1%, restricted to columns with clean unique-value ratio at most 0.5. Their alarm rate is the fraction included in the R-precision budget; zero denominators are unavailable, not zero. This reference-based mask is used only after detection and never protects rows during purification. The sensitivity analysis excluding real columns with error rate above 50% is explicitly label-dependent and exploratory, not a replacement endpoint.

For each paired contrast, draw 1,000 row-bootstrap samples with seed 2027. Use the same resampled row IDs for both arms and every run seed, retaining all cells in each selected row. Recalculate the five paired AP differences and their mean. Discard the whole bootstrap draw if any constituent seed has no positive errors, and report the valid-draw count. Percentiles give the descriptive 95% interval. These intervals condition on fitted predictions and declared seeds. They exclude retraining and new-table uncertainty, and can understate uncertainty when rows share entities. Cross-fitting and the LEO diagnostic do not make row resampling independent.

Secondary contrasts compare raw TabPFN with raw LightGBM, oracle with raw, purification with raw, raw-LEO with row-level raw, and LightGBM purification with its random control. Equal-weight macro means summarize named source tables separately for each condition. There is no pooled-cell cross-table endpoint, Wilcoxon/Holm decision or binary superiority verdict. Four sources do not become twelve independent datasets by repeating three conditions.

4.4 Prospective specification and execution integrity

The amended v2.1 specification preserves earlier protocols and discloses that descriptive dirty/clean profiles of all five tables were inspected before the amendment; only development data were used for detector debugging. Source code,

Table 1. Real benchmark universe and conditional-target support. Support percentages are specific to the conditional scorer, not universal baseline coverage.

Table	Rows	Columns	Errors	Cells supported (%)	Errors supported (%)
hospital	1000	19	509	100.0	100.0
flights	1000	6	2033	100.0	100.0
beers	1000	9	1815	88.9	100.0
rayyan	1000	10	948	50.0	84.1

input manifests, splits and environment record were frozen before confirmatory scoring, and every run records source identity, configuration, seed, checkpoint hashes and source row IDs. Interrupted and failed attempts are retained; missing seeds, unresolved partial runs, inconsistent universes or stale source hashes block the analysis. Nonfinite scores and caught fit errors keep masks and exception metadata; affected cells receive the maximum finite arm score plus one and stay in the denominator. The local environment inherits system site packages and is not isolated; version checks cover declared packages, not every transitive dependency.

5 Empirical Results

5.1 Completeness, support and primary comparison

All 60 declared table–condition–seed units completed with both models, and the saved artifacts passed input, prediction and context-boundary checks. Prediction failures occurred in one unit only, `hospital`/random injection/seed 4. There, TabPFN raised 435 caught input-validation errors (non-finite encoded values) in the final pass of every TabPFN arm and in the nested first pass; LightGBM, the other 59 units and all real-error units had none. The failures cover 54% of that unit's cells for raw, 53% for purification, 51% for random and 15% for the oracle. Following the protocol, these cells keep their place in the denominator with a tied maximal score; the raw, random and purification APs of that unit fall to about 0.11, close to its 10% error rate. Pass-1 failures receive the same penalty, so 76% of the rows removed in that unit's successful purification fits were selected by the penalty rather than by suspicion scores. Section 5.4 reports a post hoc check. A separate flights/real/seed-0 rerun reproduced every saved array and context record exactly; it is excluded from performance summaries. This is a same-implementation rerun, not an independent replication. Table 1 shows the real-data universe and conditional support.

The primary contrast compares purification with random removal on each real table, averaging paired AP differences over all five seeds. Purification was worse than random removal on three tables (hospital -0.1611, flights -0.0493, rayyan -0.0300) and better on beers ($+0.0257$); every seed had the same sign as the table mean. Table 2 gives the APs. The unweighted four-table mean is -0.0537, reported without a test or interval. The row-bootstrap intervals are

Table 2. Real-error AP, averaged over all five seeds. Final column: TabPFN purification minus matched-count random, with descriptive 95% row-bootstrap interval. Full eligible universe; intervals condition on predictions.

Table	Raw	Random	Purify	Δ [95% interval]
hospital	0.671	0.665	0.504	-0.161 [-0.202, -0.114]
flights	0.437	0.424	0.375	-0.049 [-0.057, -0.041]
beers	0.164	0.163	0.189	0.026 [0.019, 0.033]
rayyan	0.286	0.289	0.259	-0.030 [-0.039, -0.020]

Table 3. Real-error AP for measured comparators, five-seed means. DDCcore and conditional LightGBM are separate implementations, not reference DDC or CDC.

Table	VF	PF	RobustZ	Col.IF	DDCcore	LGBM
hospital	0.470	0.362	0.148	0.663	0.148	0.420
flights	0.508	0.566	0.339	0.502	0.339	0.340
beers	0.124	0.260	0.202	0.192	0.202	0.186
rayyan	0.083	0.091	0.143	0.225	0.143	0.359

descriptive: they condition on saved predictions, may be too narrow when rows share entities, and do not cover new tables, so their exclusion of zero is not inferential evidence. On real errors, therefore, the declared rule did not generally improve on removing the same number of randomly chosen rows.

5.2 Comparators and information boundary

Table 3 reports baseline APs on the same complete eligible universe. No method was best on every table. Raw TabPFN had the highest real-error AP on hospital (0.671, closely followed by column-wise Isolation Forest at 0.663), whereas pattern frequency was highest on flights and beers and conditional LightGBM on rayyan. Frequency-based methods can score targets that the conditional pipeline declares unsupported, while RobustZ and DDCcore have numerical support only. On all 20 real-error units, DDCcore returned scores identical to RobustZ (it differed in 10 of the 40 injected units). This is a defect of our frozen DDCcore code: a median of pairwise slope ratios becomes undefined when a predictor's robust score is exactly zero, which disabled the relational prediction (flights has no numerical column). We report the defect rather than correcting the confirmatory run. DDCcore and conditional LightGBM are separate implementations, not reproduced DDC or CDC systems.

Secondary differences in Table 4 distinguish conditional-model, oracle-context and observed-key boundary effects. The oracle changes context targets and conditioning features together and retains the dirty-fitted encoder, so its gap need not be positive and is not a theoretical upper bound. It was positive on every real table, indicating headroom from cleaner contexts that the declared rule did not recover. LEO uses different bases, excludes observed keys only, and keeps

Table 4. Secondary real-data AP differences and descriptive row-bootstrap intervals. LEO uses observed keys and differs from the primary row boundary; rayyan has no declared key.

Table	TFM raw − LGBM	Oracle − raw	TFM LEO − raw
hospital	0.251 [0.220, 0.277]	0.131 [0.104, 0.159]	-0.515 [-0.538, -0.493]
flights	0.097 [0.088, 0.108]	0.484 [0.458, 0.507]	-0.173 [-0.190, -0.156]
beers	-0.022 [-0.026, -0.018]	0.743 [0.733, 0.754]	-0.019 [-0.022, -0.018]
rayyan	-0.073 [-0.087, -0.060]	0.217 [0.200, 0.236]	–

Table 5. Injection probes: five-seed AP differences with descriptive row-bootstrap intervals. Conditions are repeated measurements within each source, not independent datasets.

Table	Injection	Purify − random	Oracle − raw
hospital	random	0.131 [0.123, 0.138]	0.220 [0.212, 0.229]
hospital	systematic	0.011 [0.009, 0.014]	0.844 [0.839, 0.850]
flights	random	-0.137 [-0.156, -0.118]	0.169 [0.154, 0.183]
flights	systematic	0.008 [0.003, 0.013]	0.658 [0.638, 0.678]
beers	random	0.089 [0.073, 0.104]	0.149 [0.136, 0.162]
beers	systematic	0.023 [0.018, 0.028]	0.459 [0.447, 0.473]
rayyan	random	0.031 [0.022, 0.040]	0.048 [0.041, 0.055]
rayyan	systematic	0.025 [0.019, 0.031]	0.355 [0.339, 0.374]

transductive final encoding. It lowered AP on every keyed table for both models (LightGBM: -0.283, -0.072, -0.026 on hospital, flights, beers); the large drop on hospital suggests that the row-level primary analysis benefits from same-entity rows in the context. LightGBM purification was likewise not better than its random control on real errors (-0.055, -0.006, $+0.000$ and -0.028 for hospital, flights, beers and rayyan). The leaky diagnostic, which adds the query fold to the context, changed TabPFN AP by -0.048 to $+0.046$ across tables, so self-inclusion did not uniformly inflate this score. These contrasts diagnose sensitivity rather than isolate a single causal explanation.

5.3 Injection probes and rare-valid alarms

Table 5 shows the two injection conditions. They remain separate from the real-error estimand: repeated wrong-value templates and independently selected string corruptions need not resemble natural error processes, and their seeds are coupled to context randomness. Under systematic injection, purification was slightly better than random removal on all four tables (at most $+0.025$); under random injection the effect ranged from -0.137 (flights) to $+0.131$ (hospital). The hospital random-injection value contains the failed unit. The oracle gap was larger under systematic than under random injection on every table.

Table 6 reports rare-valid alarm fractions at the reference-error-count budget, with the valid-cell denominator. This is an evaluation-only operating point.

Table 6. Rare-valid alarms in the R-precision budget on real data, five-seed means. The reference-defined denominator is shown. Rates are fractions; unavailable means no denominator.

Table	Rare-valid cells	TFM raw	Random	Purify
hospital	885	0.293	0.288	0.118
flights	1320	0.528	0.524	0.563
beers	3238	0.496	0.492	0.470
rayyan	1229	0.559	0.559	0.492

Purification lowered the alarm fraction on hospital, beers and rayyan and raised it on flights; on hospital, the reduction coincided with a large AP loss, so fewer rare-valid alarms do not by themselves indicate a better detector. No rare-valid mask was provided to purification.

5.4 Post hoc sensitivity to the failed unit

After inspecting the results, we recomputed the affected random-injection contrasts from the saved scores in two ways that were not preregistered: excluding seed 4, and evaluating seed 4 only on cells where neither compared arm failed (8,800 of 19,000 cells, error rate 9.9% versus 10.1% among failed cells). The protocol values remain the reported results. Purification minus random was +0.131 under the protocol, +0.163 over seeds 0–3 and +0.138 with failed cells dropped; the oracle gap was +0.220, +0.242 and +0.229. The sign of raw TabPFN minus raw LightGBM changed: −0.094 under the protocol, −0.0002 over seeds 0–3 and +0.016 with failed cells dropped. The failures thus do not create the hospital purification gain under random injection, but the protocol value of the model contrast in that condition reflects the failures rather than model ranking. The dropped-cells variant still uses the penalty-driven removals described above. Real-error results are unaffected.

5.5 Compute and scope of interpretation

Table 7 separates nested preparation from final purification work; the accounted matched-compute control is defined in Sect. 3. Wall times come from concurrent local CPU execution. The grid was interrupted three times and resumed without changing code, inputs or threads; incomplete attempts were retained, and some units ran while the operating system throttled background processes. Timings therefore span several execution periods: the TabPFN raw time on hospital/real, for example, ranged from 6.8 to 46.2 minutes across seeds. The table supports order-of-magnitude statements only; within a unit, nested preparation was the dominant cost of purification. Peak memory and actual neural forward-call counts were not measured.

These conclusions concern this fixed rule, the pinned weights and the capped tables; they do not show that purification generally fails or that another use of

Table 7. TabPFN time in minutes per run, five-seed means on real data. Inclusive purification equals nested preparation plus final work. Concurrent local CPU jobs; no peak-memory or forward-call measurement asserted.

Table	Raw	Random	Nested	Purify final	Purify total
hospital	30.31	27.49	68.57	9.70	78.26
flights	1.34	1.26	8.92	1.56	10.48
beers	4.45	3.92	21.19	3.90	25.08
rayyan	2.59	2.57	6.71	1.20	7.91

the same budget could not help. No outcome was used to revise the design or select seeds.

6 Limitations and Reproducibility

The collection contains four capped source tables, so per-table comparisons can expose heterogeneous effects but not population-wide superiority. Descriptive data access preceded the amended specification, which limits claims of an untouched benchmark. Exact reference disagreement includes formatting changes and possible ambiguity in benchmark corrections. Our corruptions start from the clean reference, may be easier than natural errors and share the seed with context sampling; they are mechanism probes, not substitutes for the real-error experiment. The oracle changes all context features, so its gap does not isolate target-cell contamination.

The final encoder observes all dirty rows, and the primary analysis allows other records of the same entity in the context. Observed-key LEO cannot repair key errors or stop the encoder from seeing held-out rows. Row-bootstrap intervals may be too narrow under dependence, omit retraining and new-table variation, and are not multiplicity-controlled.

Numerical tail scores and categorical probabilities are not proven equally calibrated. LightGBM treats frequency-rank codes as numerical covariates, whereas TabPFN receives categorical indices, so the model contrast also reflects feature handling. Marginal baselines use whole observed columns, while conditional models exclude query folds. Whole-universe penalties keep unsupported and failed cells in the denominator, but do not make support equivalent; in one injected unit the failure penalty covered about half of all TabPFN cells and reversed one secondary contrast. Rare-valid alarm rates cover a reference-defined subset and do not establish protection of legitimate minority values.

Purification is one suspicion-based rule at one removal fraction and context cap; it is neither an influence function nor a robustness theorem. The accounted matched-compute control does not test whether another use of the same compute, such as context ensembling, would help. Other selection strategies, context-size sweeps and clean-calibration regimes are outside this study. DDCcore coincided with RobustZ on every real-error unit and adds no evidence about relational

numerical detection. Reference CDC, reference DDC, unsupervised TabPFN extensions, Auto-Test, MaTElDa and LUCARIO were not run, which precludes state-of-the-art claims.

Reproduction requires the pinned data and weights, the source-freeze manifest, declared package versions and the retained predictions. The analysis script recreates metrics, intervals, tables and figures from those predictions; rerunning it gave byte-identical outputs, and a separate audit checks input identity, context-boundary logs and AP against scikit-learn. These checks reuse our own implementation and are not an independent replication. Code and checkpoint licenses are separate, and redistribution must respect the pinned weight terms. Built with PriorLabs-TabPFN.

7 Conclusion

We evaluated a simple context-purification intervention for conditional cell-error scoring under an explicitly transductive row-level protocol. On real errors, removing the most suspicious context rows was worse than removing the same number of random rows on three of the four tables, although a clean-context oracle showed room for improvement on every table; the declared rule did not exploit that headroom in general. Nested isolation, complete denominators, failure masks and explicit preparation cost were needed to interpret these results, which provide neither a robustness theorem nor a cross-dataset superiority claim. Broader conclusions require more datasets, other context strategies, direct cell-level competitors and dependence-aware validation.

Use of AI tools. AI coding and writing assistants (OpenAI Codex and Anthropic Claude) were used for literature search, implementation, test and audit scripts, experiment orchestration, analysis checks, and drafting and editing of this text. The author takes responsibility for the design decisions and for all sources, numbers and statements in the paper; no AI system is an author.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Ahmadi, F., Speckmann, M., Kuhlmann, M.F., Abedjan, Z.: MaTElDa: Multi-table error detection. In: Proceedings of the 28th International Conference on Extending Database Technology (EDBT). pp. 364–376 (2025). https://doi.org/10.48786/edbt.2025.29
2. Berti-Equille, L.: Model-aware data cleaning for tabular foundation models. arXiv preprint arXiv:2604.25154v2 (2026), https://arxiv.org/abs/2604.25154v2
3. Chen, Q., He, Y., Wong, R.C.W., Cui, W., Ge, S., Zhang, H., Zhang, D., Chaudhuri, S.: Auto-test: Learning semantic-domain constraints for unsupervised error detection in tables. Proceedings of the ACM on Management of Data **3**(3), 133:1–133:27 (2025). https://doi.org/10.1145/3725396

4. Fibbi, E., Perrotta, D., Torti, F., Van Aelst, S., Verdonck, T.: Cell-TRICC: A model-based approach to cellwise-trimmed co-clustering. Journal of Computational and Graphical Statistics (2026). https://doi.org/10.1080/10618600.2026.2634828, published online 21 April 2026
5. Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S.B., Schirrmeister, R.T., Hutter, F.: Accurate predictions on small data with a tabular foundation model. Nature **637**, 319–326 (2025). https://doi.org/10.1038/s41586-024-08328-6
6. Ipas, O., Gomez-Trenado, G., Romero-Zaliz, R., Triguero, I.: LUCoS: Latent unsupervised context selection for tabular foundation models. arXiv preprint arXiv:2605.27254v1 (2026), https://arxiv.org/abs/2605.27254v1
7. Jäger, S., Biessmann, F.: From data imputation to data cleaning — automated cleaning of tabular data improves downstream predictive performance. In: Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 238, pp. 3394–3402 (2024), https://proceedings.mlr.press/v238/jager24a.html
8. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
9. Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation forest. In: 2008 Eighth IEEE International Conference on Data Mining. pp. 413–422 (2008). https://doi.org/10.1109/ICDM.2008.17
10. Mahdavi, M., Abedjan, Z., Castro Fernandez, R., Madden, S., Ouzzani, M., Stonebraker, M., Tang, N.: Raha: A configuration-free error detection system. In: Proceedings of the 2019 International Conference on Management of Data (SIGMOD). pp. 865–882 (2019). https://doi.org/10.1145/3299869.3324956
11. Marszałek, P., Kuśmierczyk, T., Śmieja, M.: TACTIC for navigating the unknown: Tabular anomaly detection via in-context inference. arXiv preprint arXiv:2603.14171 (2026), https://arxiv.org/abs/2603.14171
12. Ni, W., Zhang, K., Miao, X., Zhao, X., Wu, Y., Wang, Y., Yin, J.: ZeroED: Hybrid zero-shot error detection through large language model reasoning. arXiv preprint arXiv:2504.05345 (2025), https://arxiv.org/abs/2504.05345
13. Ning, K., Biloš, M., Wilson, J.T., Zhang, Y., Rasul, K., Song, D., Schneider, A., Nevmyvaka, Y.: Test-time compute for tabular foundation models: Mechanisms, gains, and limits. arXiv preprint arXiv:2610.12005v1 (2026), https://arxiv.org/abs/2610.12005v1
14. Papastergios, V., Gounaris, A.: Out in the wild: Investigating the impact of imperfect data on a tabular foundation model. In: 14th International Workshop on Quality in Databases (QDB) (2025), https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/QDB/QDB25_1.pdf
15. Qin, Q., Li, H., Merlo, E.: Unsupervised, robust, and lightweight detection of data pattern anomalies and outliers. Information and Software Technology **192**, 108035 (2026). https://doi.org/10.1016/j.infsof.2026.108035
16. Rousseeuw, P.J., Van den Bossche, W.: Detecting deviating data cells. Technometrics **60**(2), 135–145 (2018). https://doi.org/10.1080/00401706.2017.1340909
17. Zaccaria, G., García-Escudero, L.A., Greselin, F., Mayo-Íscar, A.: Cellwise outlier detection in heterogeneous populations. arXiv preprint arXiv:2409.07881v3 (2025), https://arxiv.org/abs/2409.07881v3